Bank Customer Churn Prediction & Risk Scoring

Research Report

Four-model comparison, risk estimation, explainability and a Streamlit dashboard

Project type: Educational / internship project
Models: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting
Deployed model: Random Forest classifier
Interface: Streamlit dashboard

Prepared from the project source files and results re-run on the supplied dataset. Findings are limited to the available evidence and are not a production validation.

Abstract

Customer churn affects revenue stability and long-term customer relationships. This project builds a supervised machine-learning workflow that estimates the probability that a bank customer exits. Four classifiers are compared on a stratified 80/20 holdout split of 10,000 customers: Logistic Regression, Decision Tree, Random Forest and Gradient Boosting, with 5-fold cross-validated ROC-AUC as a stability check. Gradient Boosting achieved the highest accuracy, precision, F1-score and ROC-AUC, Random Forest offered a more balanced precision/recall profile, and the Decision Tree identified the most churners at the cost of many false alarms. Explainability is provided through Random Forest feature importance, SHAP values and partial dependence plots, and the results are delivered through a Streamlit dashboard with a risk calculator, probability distribution, feature importance, model comparison and a what-if simulator. The work is an educational prototype; data provenance, calibration, threshold selection, fairness and external validation require further study.

1. Introduction

Customer churn prediction is a binary classification task: estimate whether a customer will leave using profile and relationship variables. A probability score allows a bank to prioritise retention effort before customers leave, rather than reacting afterwards. A probability does not prove that a customer will leave or explain why. A useful system therefore reports missed churners and false alarms, offers explanations, and states its limits.

2. Objectives

- Inspect the dataset and the target distribution.
- Prepare customer attributes and engineer features for supervised learning.
- Compare Logistic Regression, Decision Tree, Random Forest and Gradient Boosting using accuracy, precision, recall, F1-score and ROC-AUC.
- Check stability with 5-fold cross-validation.
- Explain the model with feature importance, SHAP and partial dependence.
- Provide a dashboard for customer-level risk estimates and what-if scenarios.

3. Dataset and exploratory findings

The supplied CSV contains 10,000 rows and 14 columns. The target field is *Exited* (0 = stayed, 1 = exited). No missing values and no duplicate CustomerId values were found. The *Year* column holds the single value 2025 for every row, so it carries no information and is dropped.

Target class	Count	Share
Stayed (0)	7,963	79.63%
Exited (1)	2,037	20.37%

Observed churn rates by group are shown below. These are associations within this dataset and do not establish causes.

Group	Customers	Observed churn rate
France	5,014	16.15%
Germany	2,509	32.44%
Spain	2,477	16.67%
Inactive member	4,849	26.85%
Active member	5,151	14.27%
Female	4,543	25.07%
Male	5,457	16.46%
1 product	5,084	27.71%
2 products	4,590	7.58%
3 products	266	82.71%
4 products	60	100.00%

Small-group caveat. Only 266 customers hold three products and only 60 hold four. The very high churn rates in these groups (82.71% and 100%) rest on small samples and should be verified against the data source before any business conclusion is drawn. Average age was 37.41 for customers who stayed and 44.84 for those who exited.

4. Data preparation and feature engineering

CustomerId and Surname are removed as identifiers, and Year is removed because it is constant. Geography and Gender are one-hot encoded with OneHotEncoder (handle_unknown="ignore"). Numerical fields are standardised with StandardScaler inside a scikit-learn ColumnTransformer and Pipeline. A stratified 80/20 split with random_state=42 gives 8,000 training and 2,000 test records and preserves the churn ratio.

Derived feature	Definition
BalanceSalaryRatio	Balance / EstimatedSalary (zero salary guarded)
ProductDensity	NumOfProducts / (Tenure + 1)
EngagementProduct	IsActiveMember × NumOfProducts
AgeTenureInteraction	Age × Tenure

The feature-engineering logic is repeated across scripts and the dashboard. Centralising it would reduce the risk of training/serving mismatch.

5. Models and evaluation

Four models are trained on the same training split and evaluated on the same 2,000-record test set. Logistic Regression is the interpretable baseline. The Decision Tree (max_depth=6, min_samples_leaf=20, balanced class weights) is a transparent tree-based model. The Random Forest uses 200 trees with balanced class weights. Gradient Boosting uses scikit-learn defaults. XGBoost, which is optional in the project brief, was not run. Precision measures how many flagged customers actually left, recall measures how many leavers were found, and ROC-AUC measures ranking quality across thresholds.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV AUC (5-fold)
Logistic Regression	0.8060	0.5704	0.1892	0.2841	0.7735	0.7625 ± 0.0203
Decision Tree	0.7620	0.4513	0.7862	0.5735	0.8410	0.8323 ± 0.0080
Random Forest	0.8410	0.6121	0.5971	0.6045	0.8522	0.8470 ± 0.0124
Gradient Boosting	**0.8705**	**0.7913**	**0.4939**	**0.6082**	**0.8674**	**0.8631 ± 0.0093**

Test-set metrics use the default 0.50 threshold. The last column is the mean ± standard deviation of ROC-AUC across five stratified folds of the training data.

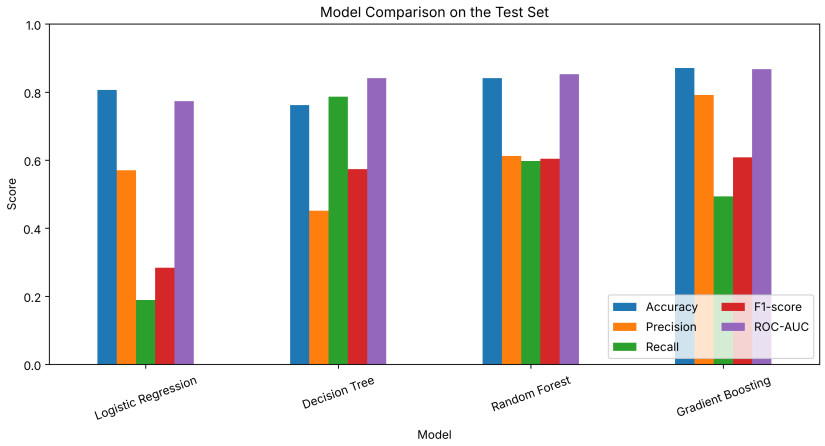


Figure 1. Test-set metrics for the four models.

Reading the results

- **Gradient Boosting** scored highest on accuracy (0.8705), precision (0.7913), F1 (0.6082) and ROC-AUC (0.8674), and also had the highest cross-validated AUC. Its F1 advantage over Random Forest is small (0.6082 against 0.6045).
- **Random Forest** has higher recall than Gradient Boosting (0.5971 against 0.4939), so it finds more churners while keeping precision above 0.61.
- **Decision Tree** has the highest recall (0.7862) but low precision (0.4513): it catches most leavers while raising many false alarms.
- **Logistic Regression** has acceptable accuracy but misses over 80% of churners (recall 0.1892), mainly because churn is only about 20% of the data and the model does not use class weighting.
- Cross-validated AUC is close to test-set AUC for every model, so the holdout result does not look like an unusually lucky split.

The dashboard uses the Random Forest model. It was kept as the deployed model because it balances recall and precision and has an established explanation workflow (feature importance, SHAP, partial dependence). Gradient Boosting is a strong candidate for a future version.

6. Confusion matrix and risk trade-offs

For the Random Forest at the 0.50 threshold (rows = actual, columns = predicted):

	Predicted stayed	Predicted exited
Actual stayed	1,439 (TN)	154 (FP)
Actual exited	164 (FN)	243 (TP)

The model identified 243 of the 407 customers who exited and missed 164. It also flagged 154 customers who stayed. A lower threshold would raise recall at the cost of more false alarms. The threshold should be tuned on validation data using the cost of a missed churner versus the cost of unnecessary retention outreach, not chosen to maximise accuracy.

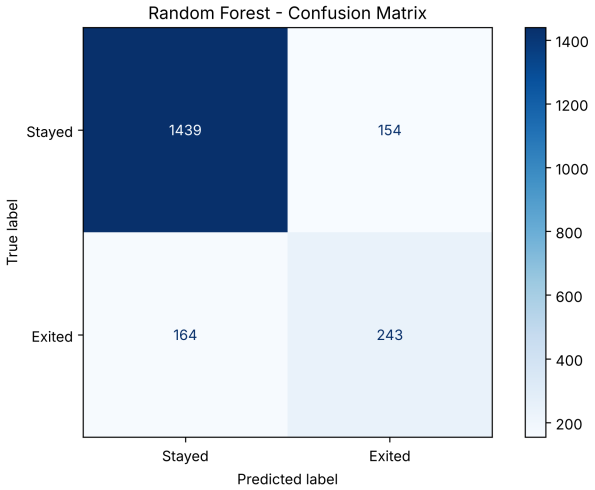


Figure 2. Random Forest confusion matrix on the test set.

7. Explainability

Three techniques describe how the Random Forest uses its inputs. None of them proves cause and effect.

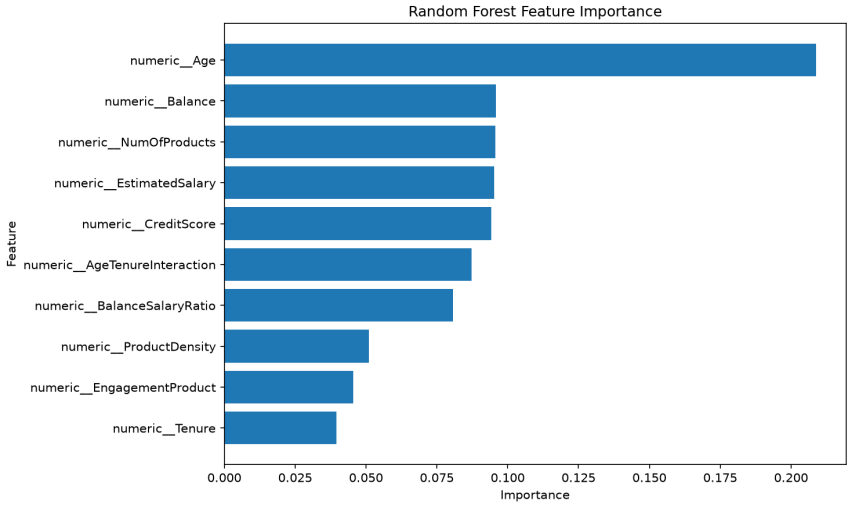


Figure 3. Random Forest feature importance.

The SHAP summary plot below is based on a sample of up to 200 transformed test rows. SHAP values attribute each prediction to the input features under the explainer's assumptions.

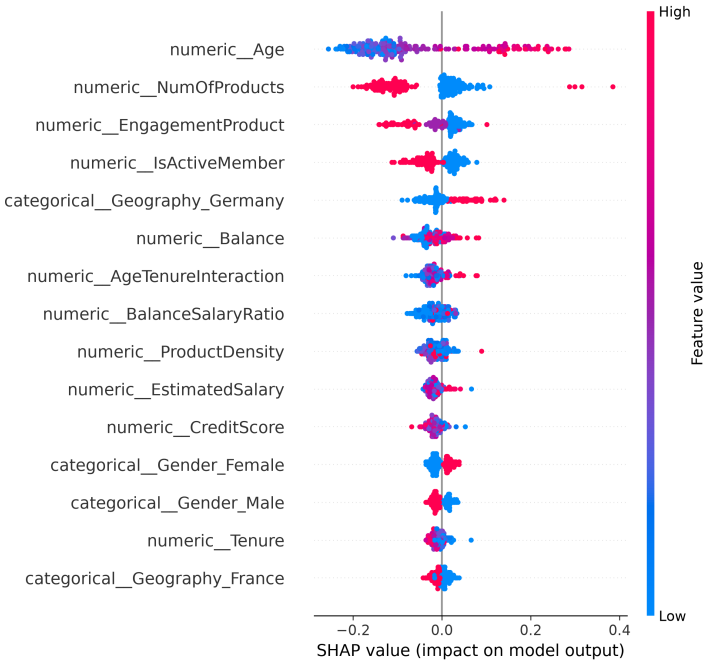


Figure 4. SHAP summary plot for the transformed test sample.

7.1 Partial dependence

Partial dependence plots show the average predicted churn probability as one feature is varied while the others keep their observed values. Figure 5 was computed with the trained Random Forest on a random sample of 2,000 customers.

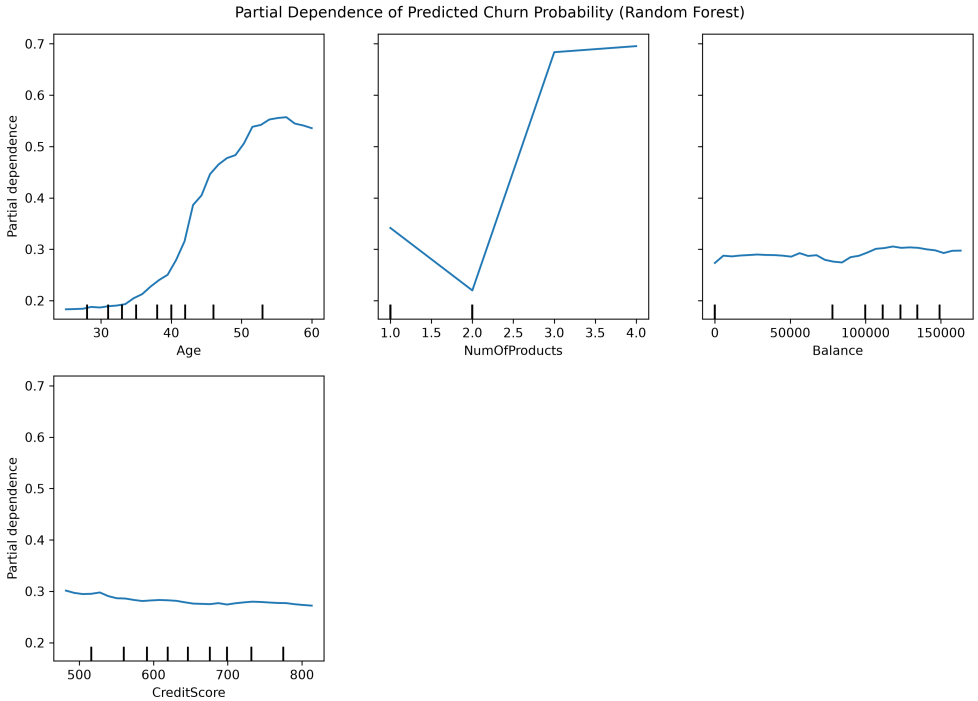


Figure 5. Partial dependence of predicted churn probability for Age, NumOfProducts, Balance and CreditScore.

- **Age:** the average predicted probability is roughly 0.18–0.20 below about age 33, rises steeply through the 40s, and reaches about 0.55 near age 55 before levelling off.
- **NumOfProducts:** predicted risk is about 0.34 for one product, falls to about 0.22 for two, and jumps to about 0.68–0.70 for three or four. This follows the descriptive rates in Section 3, and the three- and four-product groups are small.

- **Balance:** the curve is nearly flat (about 0.27–0.31), so balance on its own has little average effect in this model.
- **CreditScore:** a slight decline from about 0.30 to 0.27 across the range, a weak average effect.

Lines between the product counts are interpolation for display; only whole numbers of products exist. Partial dependence averages over other features and can hide interactions, so it should be read as a description of model behaviour rather than a guide to what would change a real customer's decision.

8. Streamlit dashboard

The dashboard (app.py) provides the modules requested in the project brief:

- **Customer churn risk calculator:** sidebar inputs for all customer attributes produce a churn probability and a label at the fixed 0.50 threshold.
- **Probability distribution:** histogram of model-estimated churn probabilities, grouped by actual outcome, drawn from a pre-computed scores file (models/scored_customers.csv) that holds only the score and outcome, with no customer details.
- **Feature importance dashboard:** Random Forest importance, SHAP summary, model comparison and partial dependence charts.
- **What-if scenario simulator:** changes activity status and number of products and shows the original probability, the scenario probability and the difference.
- **Confusion matrix** for the Random Forest on the test set.

The scenario simulator is a model-based sensitivity demonstration, not evidence that changing a customer attribute would change real churn. Because the scored file is computed from the full dataset, including the records used for training, its probability separation looks stronger than it would on unseen customers. Run with: `python -m streamlit run app.py`.

9. Recommendations

- Verify the dataset origin, licence and permission to redistribute before public release.
- Tune the decision threshold on validation data using the relative costs of false negatives and false positives.
- Consider Gradient Boosting (and optionally XGBoost) with hyperparameter tuning, and calibrate predicted probabilities.
- Add independent temporal or external validation beyond a single holdout and cross-validation.
- Audit performance across customer groups, especially by geography and gender, before any real-world use.
- Centralise preprocessing and feature engineering so training and dashboard inference share one implementation.
- Use the dashboard to support analyst review only; do not make consequential decisions from these predictions alone.

10. Limitations

Results come from one supplied dataset, one holdout split and a single cross-validation configuration measured only by ROC-AUC. Precision, recall and F1 were not cross-validated. Hyperparameters were not tuned, and probability calibration, threshold optimisation, fairness and external validation were not assessed. Churn rates for the three- and four-product groups rest on 266 and 60 customers. Model explanations describe model behaviour, not causal mechanisms. The application loads a pickle file, which should only be loaded from a trusted source.

11. Conclusion

This project delivers an end-to-end churn prediction workflow: data inspection, feature engineering, a four-model comparison with cross-validation, three explainability techniques and an interactive Streamlit dashboard. Gradient Boosting gave the best overall ranking and precision; Random Forest, the deployed model, gave a more balanced catch rate; and all tree-based models clearly outperformed the Logistic Regression baseline on recall. Missed churners and false alarms remain, so further validation, threshold analysis, calibration, fairness review and data governance are needed before any real-world use.

Appendix: Reproducibility commands

```
python inspect_data.py
python eda.py
python feature_analysis.py
python compare_models.py
python feature_importance.py
python shap_explainability.py
python partial_dependence.py
python make_scores.py
python -m streamlit run app.py
```

These commands assume the authorised dataset is at data/churn.csv and the dependencies in requirements.txt are installed (Python 3.12).

Disclaimer

This report is for educational and portfolio purposes only. The prototype is not validated for production banking use, and its predictions are not guarantees of future behaviour.